

AT32基于mbed TLS的HTTPS服务器

前言

传统的 HTTP 使用明文传输，有心拦截传输数据的第三方都可以透过侧录封包知道传输的内容，在这样的情况下传输的数据缺乏保障，若内容涉及个人隐私及银行的账号密码，将使个人蒙受巨大的损失；为了保障数据传输的安全，HTTPS 应运而生，透过对传输双方的身份认证，将传输的内容加密，使得恶意的第三方无法得知实际上传输的内容为何。本应用笔记将介绍如何使用 mbed TLS 搭建一个 HTTPS 服务器，用户可以根据自己的应用去编写网页内容的同时，又简单地将传输数据加密。

支持型号列表：

支持型号	AT32F407xx
	AT32F437xx

目录

1	HTTPS 概述	5
2	例 HTTPS 服务器	7
2.1	功能简介	7
2.2	资源准备	7
2.3	软件设计	7
2.3.1	使用 OpenSSL 建立自签凭证	15
2.4	实验效果	16
3	文档版本历史	18

表目录

表 1. 文档版本历史	18
-------------------	----

图目录

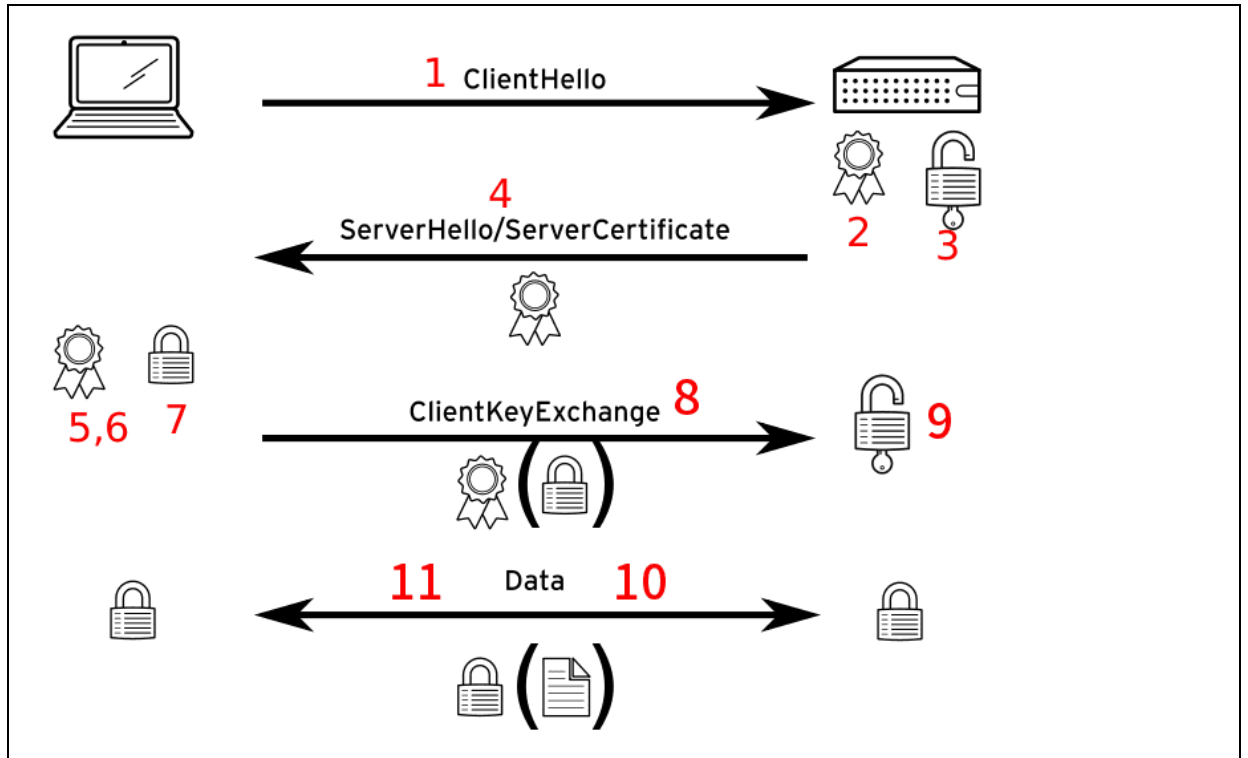
图 1. HTTPS 原理.....	6
图 2. 透过 Run-Time 管理器加入 mbed TLS	7
图 3. HTTPS 页面.....	17

1 HTTPS 概述

HTTPS 的安全性是基于 Transport Layer Security (TLS), TLS 是一种网络加密通信的方式, 作为 Secure Sockets Layer (SSL) 的接续协议, TLS 允许客户端与服务端的互相验证。TLS 以凭证为概念, 凭证包含: 公钥、服务器身份、凭证颁发单位的签名。对应的私钥永远不会公开, 任何使用私钥加密的密钥数据只能用公钥来解密, 反之亦然。整个加密通信流程, 可以透过图 1 来简单描述:

1. 客户端发起 hello 握手: 给服务器的讯息有:
 - 包含时间戳的 32 位随机数字 `client_random`
 - 加密协定
 - 客户端支持的加密方式
2. 服务端必须要有一套证书, 可以自制或向组织申请。自己颁发的证书需要客户端验证通过, 才可以继续访问, 使用受信任单位申请的证书不会弹出提示页面。
3. 一对公钥和私钥, 可以想象成一把钥匙 (私钥) 和一个锁头 (公钥), 把锁头给客户端将重要的数据锁起来, 客户端将锁好的数据传给服务器, 只有服务器有开锁的钥匙可以解开, 所以即使传送过程被截取也无法破解。
4. 对于客户端的 hello 握手, 服务端响应以下讯息给客户端:
 - 另一个包含时间戳的 32 位随机数字 `server_random`
 - 加密协定
 - 加密方式
 - 服务器证书: 包含拥有者名称、网站地址、证书公钥、证书颁发机构数字签名、过期时间等。
5. 客户端验证服务器传来的凭证是否有效? 例如颁发机构、过期时间等, 如果发现异常, 则会弹出一个警告框, 提示证书存在问题。(本应用笔记不是使用第三方证书授权中心 (CA) 签发的凭证, 而是使用自己颁发的凭证, 所以客户端必须先取得签发机构的公钥(下一节的 `kvm5.pem`) 来验证颁发机构签名, 才不会弹出警告框)
6. 在此之前的所有 TLS 握手讯息都是明文传送, 现在收到服务器的证书且验证没问题, 则客户端先产生 `PreMaster_Secret`
 - 使用加密算法, 例如: RSA, Diffie-Hellman, 对 `server_random` 运算产生。
 - 或称 `PreMaster_Key`
 - 是一个 48 个位的 Key, 前 2 个字节是协议版本号, 后 46 字节是用在对称加密密钥的随机数字。
7. 客户端用服务端送来的公钥加密 `PreMaster_Secret`。
8. 客户端将加密的 `PreMaster_Secret` 传送给服务端, 目的是让服务端可以像客户端用一样随机值产生 `Master_Secret`。
9. 服务端用私钥解密 `PreMaster_Secret`.
 - 此时客户端与服务端都有一份相同的 `PreMaster_Secret` 和随机数 `client_random`, `server_random`。
 - 使用 `client_random` 及 `server_random` 当种子, 结合 `PreMaster_Secret`, 客户端和服务端将计算出同样的 `Master_Secret`。
 - 作为资料加解密相关的 Key Material。
10. 作为资料加解密相关的 Key Material。
11. 作为资料加解密相关的 Key Material。

图 1. HTTPS 原理

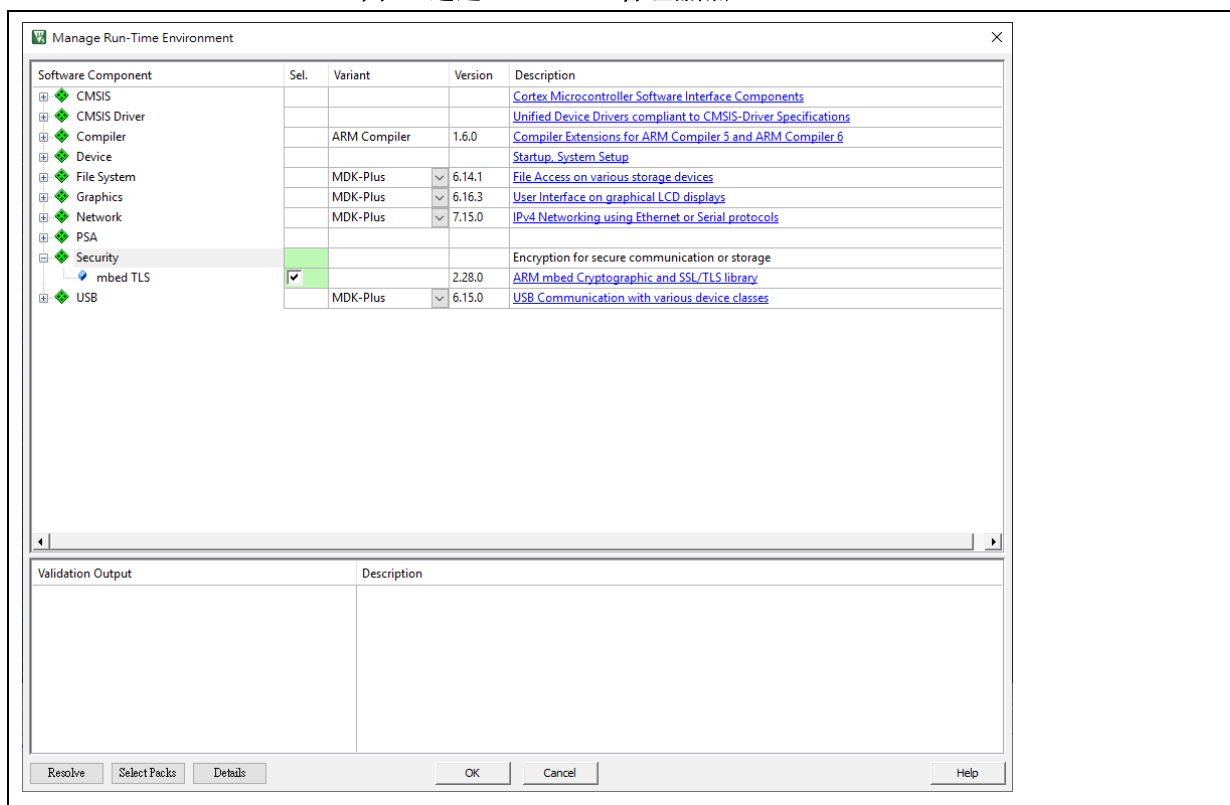


2 例 HTTPS 服务器

2.1 功能简介

本示例需要使用到 EMAC 的功能，搭配 LwIP 协议栈，此协议栈有提供 HTTPS 的 API，但会使用到 mbed TLS 的加密功能，这包库从 Keil 的官方网站或是透过 Keil 内的 Pack Installer 可以获得。

图 2. 透过 Run-Time 管理器加入 mbed TLS



2.2 资源准备

- 1) 硬件环境:
对应产品型号的 AT-START BOARD
- 2) 软件环境
at32f4xx\project\mdk_v5

2.3 软件设计

- 1) 配置流程
 - 配置 EMAC 功能
 - 初始化 LwIP 协议栈
 - 配置私钥及 CA 凭证给服务器
 - 初始化 HTTPS 服务器
- 2) 代码介绍
 - ssl_server 函数代码描述

```
void ssl_server(void const *argument)
{
    int ret, len;

#ifdef MBEDTLS_MEMORY_BUFFER_ALLOC_C
    mbedtls_memory_buffer_alloc_init(memory_buf, sizeof(memory_buf));
#endif

    //mbedtls_net_init( &listen_fd );
    listen_fd.fd = -1;
    //mbedtls_net_init( &client_fd );
    client_fd.fd = -1;
    mbedtls_ssl_init( &ssl );
    mbedtls_ssl_config_init( &conf );
#ifdef MBEDTLS_SSL_CACHE_C
    mbedtls_ssl_cache_init( &cache );
#endif

    mbedtls_x509_crt_init( &srvcert );
    mbedtls_pk_init( &pkey );
    mbedtls_entropy_init( &entropy );
    mbedtls_ctr_drbg_init( &ctr_drbg );

#ifdef MBEDTLS_DEBUG_C
    mbedtls_debug_set_threshold( DEBUG_LEVEL );
#endif

    /*
     * 1. Load the certificates and private RSA key
     */
    mbedtls_printf( "\r\n  . Loading the server cert. and key..." );

    /*
     * This demonstration program uses embedded test certificates.
     * Using mbedtls_x509_crt_parse_file() to read the server and CA certificates
     * requires the implementation of the File I/O API using the FatFs, that is
     * not implemented yet.
     */
}
```



```

    ret = mbedtls_x509_crt_parse( &srvcert, (const unsigned char *) mbedtls_test_srv_crt,
mbedtls_test_srv_crt_len );
    if(ret != 0)
    {
        mbedtls_printf( " failed\r\n  !  mbedtls_x509_crt_parse returned %d\r\n", ret );
        goto exit;
    }

    ret =  mbedtls_pk_parse_key(&pkey, (const unsigned char *) mbedtls_test_srv_key,
mbedtls_test_srv_key_len, NULL, 0);

    if( ret != 0 )
    {
        mbedtls_printf(" failed\r\n  !  mbedtls_pk_parse_key returned %d\r\n", ret);
        goto exit;
    }

    mbedtls_printf( " ok\r\n" );

    /*
    * 2. Setup the listening TCP socket
    */
    mbedtls_printf( "   . Bind on https://localhost:443/ ..." );

    if((ret = mbedtls_net_bind(&listen_fd, NULL, "443", MBEDTLS_NET_PROTO_TCP )) != 0)
    {
        mbedtls_printf( " failed\r\n  ! mbedtls_net_bind returned -0x%x\r\n", -ret );
        goto exit;
    }

    mbedtls_printf( " ok\r\n" );

    /*
    * 3. Seed the RNG
    */
    mbedtls_printf( "   . Seeding the random number generator..." );

    if((ret = mbedtls_ctr_drbg_seed(&ctr_drbg, ALTCP_MBEDTLS_RNG_FN, &entropy, (const

```

```
unsigned char *) pers, strlen( (char *)pers))) != 0)
{
    mbedtls_printf( " failed\r\n  ! mbedtls_ctr_drbg_seed returned -0x%x\r\n", -ret );
    goto exit;
}

mbedtls_printf( " ok\r\n" );

/*
 * 4. Setup stuff
 */
mbedtls_printf( " . Setting up the SSL data...." );

if((ret = mbedtls_ssl_config_defaults(&conf, MBEDTLS_SSL_IS_SERVER,
MBEDTLS_SSL_TRANSPORT_STREAM, MBEDTLS_SSL_PRESET_DEFAULT)) != 0)
{
    mbedtls_printf( " failed\r\n  ! mbedtls_ssl_config_defaults returned -0x%x\r\n", -ret );
    goto exit;
}

mbedtls_ssl_conf_rng(&conf, mbedtls_ctr_drbg_random, &ctr_drbg);

#ifdef MBEDTLS_SSL_CACHE_C
    mbedtls_ssl_conf_session_cache(&conf, &cache, mbedtls_ssl_cache_get,
mbedtls_ssl_cache_set);
#endif

mbedtls_ssl_conf_ca_chain(&conf, srvcert.next, NULL);
if((ret = mbedtls_ssl_conf_own_cert(&conf, &srvcert, &pkey)) != 0)
{
    mbedtls_printf( " failed\r\n  ! mbedtls_ssl_conf_own_cert returned -0x%x\r\n", -ret );
    goto exit;
}

if((ret = mbedtls_ssl_setup(&ssl, &conf)) != 0)
{
    mbedtls_printf( " failed\r\n  ! mbedtls_ssl_setup returned -0x%x\r\n", -ret );
    goto exit;
}
```

```
}

mbedtls_printf( " ok\r\n" );

reset:
#ifdef MBEDTLS_ERROR_C
    if(ret != 0)
    {
        uint8_t error_buf[100];
        mbedtls_strerror( ret, (char *)error_buf, 100 );
        mbedtls_printf("Last error was: -0x%x - %s\r\n", -ret, error_buf );
    }
#endif

    mbedtls_net_free(&client_fd);

    mbedtls_ssl_session_reset(&ssl);

    /*
     * 5. Wait until a client connects
     */
    mbedtls_printf( " . Waiting for a remote connection ...\r\n" );

    if((ret = mbedtls_net_accept(&listen_fd, &client_fd, NULL, 0, NULL)) != 0)
    {
        mbedtls_printf(" => connection failed\r\n ! mbedtls_net_accept returned -0x%x\r\n", -ret);
        goto exit;
    }

    mbedtls_ssl_set_bio(&ssl, &client_fd, mbedtls_net_send, mbedtls_net_recv, NULL);

    mbedtls_printf(" => connection ok\r\n");

    /*
     * 6. Handshake
     */
    mbedtls_printf(" . Performing the SSL/TLS handshake...");
```

```
while((ret = mbedtls_ssl_handshake(&ssl)) != 0)
{
    if(ret != MBEDTLS_ERR_SSL_WANT_READ && ret !=
MBEDTLS_ERR_SSL_WANT_WRITE)
    {
        mbedtls_printf(" failed\r\n  ! mbedtls_ssl_handshake returned -0x%x\r\n", -ret);
        goto reset;
    }
}

mbedtls_printf(" ok\r\n");

/*
 * 7. Read the HTTP Request
 */
mbedtls_printf(" < Read from client:");
do
{
    len = sizeof(buf) - 1;
    memset(buf, 0, sizeof(buf));
    ret = mbedtls_ssl_read(&ssl, buf, len);

    if(ret == MBEDTLS_ERR_SSL_WANT_READ || ret ==
MBEDTLS_ERR_SSL_WANT_WRITE)
    {
        continue;
    }
    if(ret <= 0)
    {
        switch(ret)
        {
            case MBEDTLS_ERR_SSL_PEER_CLOSE_NOTIFY:
                mbedtls_printf(" connection was closed gracefully\r\n");
                break;

            case MBEDTLS_ERR_NET_CONN_RESET:
                mbedtls_printf(" connection was reset by peer\r\n");
                break;
```

```
        default:
            mbedtls_printf(" mbedtls_ssl_read returned -0x%x\r\n", -ret);
            break;
    }

    break;
}

len = ret;
mbedtls_printf(" %d bytes read\r\n%s", len, (char *) buf);

if(ret > 0)
{
    break;
}
} while(1);

/*
 * 8. Write the 200 Response
 */
mbedtls_printf(" > Write to client:");
len = sprintf((char *) buf, HTTP_RESPONSE, mbedtls_ssl_get_ciphersuite(&ssl));

while((ret = mbedtls_ssl_write(&ssl, buf, len)) <= 0)
{
    if(ret == MBEDTLS_ERR_NET_CONN_RESET)
    {
        mbedtls_printf(" failed\r\n ! peer closed the connection\r\n");
        goto reset;
    }

    if(ret != MBEDTLS_ERR_SSL_WANT_READ && ret !=
MBEDTLS_ERR_SSL_WANT_WRITE)
    {
        mbedtls_printf(" failed\r\n ! mbedtls_ssl_write returned -0x%x\r\n", -ret);
        goto exit;
    }
}
```

```
}

len = ret;
mbedtls_printf(" %d bytes written\r\n%s", len, (char *) buf);

mbedtls_printf(" . Closing the connection...");

while((ret = mbedtls_ssl_close_notify(&ssl)) < 0)
{
    if(ret != MBEDTLS_ERR_SSL_WANT_READ && ret !=
MBEDTLS_ERR_SSL_WANT_WRITE)
    {
        mbedtls_printf(" failed\r\n ! mbedtls_ssl_close_notify returned -0x%x\r\n", -ret );
        goto reset;
    }
}

mbedtls_printf(" ok\r\n");

ret = 0;
goto reset;

exit:
mbedtls_net_free( &client_fd );
mbedtls_net_free( &listen_fd );

mbedtls_x509_crt_free( &srvcert );
mbedtls_pk_free( &pkey );
mbedtls_ssl_free( &ssl );
mbedtls_ssl_config_free( &conf );
#if defined(MBEDTLS_SSL_CACHE_C)
mbedtls_ssl_cache_free( &cache );
#endif
mbedtls_ctr_drbg_free( &ctr_drbg );
mbedtls_entropy_free( &entropy );

while(1);
}
```

2.3.1 使用 OpenSSL 建立自签凭证

在本应用笔记中，我们将使用自签凭证来建立 TLS 联机，而发行自签凭证会使用到 OpenSSL 这个工具，以下会简单介绍在 Windows 上及在 Linux Ubuntu 上，如何安装 OpenSSL。

- Windows

因为 OpenSSL 未提供可执行的安装档，因此我们透过 [Git for Windows](#) 来达到安装

OpenSSL 的目的；当安装完成后，默认执行文件路径为 C:\Program

Files\Git\usr\bin\openssl.exe，你可以将 C:\Program Files\Git\usr\bin 路径加入到 PATH 环境变量之中，以后就可以直接输入 openssl 来执行此工具。

- Ubuntu

只需要在终端机中下命令

```
sudo apt install openssl
```

在确定 PC 上有 OpenSSL 这个工具后，基本上只要按照以下步骤，就一定可以建立出合法的自签凭证：

1. 建立 ssl.conf 配置文件

```
[req]
prompt = no
default_md = sha256
default_bits = 2048
distinguished_name = dn
x509_extensions = v3_req

[dn]
C = KY
ST = Cayman Islands
L = George Town
O = ARTERY
OU = SW Department
emailAddress = admin@example.com
CN = localhost

[v3_req]
subjectAltName = @alt_names

[alt_names]
DNS.1 = *.localhost
DNS.2 = localhost
IP.1 = 172.31.96.101
```

上述配置文件内容的 [dn] 区段 (Distinguished Name) 为凭证的相关信息，你可以自由调整为你想设定的内容，其中 O (Organization) 是公司名称，OU (Organization Unit) 是部门名称，而 CN (Common Name) 则是凭证名称，你可以设定任意名称，设定中文也可以，但请记得档案要以 UTF-8 编码存盘，且不能有 BOM 字符。

配置文件的 [alt_names] 区段，则是用来设定 SSL 凭证的域名，这部分设定相当重要，如果没有设定的话，许多浏览器都会将凭证视为无效凭证。这部分你要设定几组域名都可以，基本上没有什么上限，因为自签凭证主要目的是用来开发测试之用，因此建议可以把可能会用到的本机域名 (localhost) 或是局域网络的 IP 地址都加上去，以便后续进行远程联机测试。

2. 打开**终端机工具**后，切换到存放 ssl.conf 的目录下后，透过 OpenSSL 命令产生出自签凭证与相对应的私钥，输入以下命令就可以建立出 私钥 (server.key) 与 凭证档案 (server.crt)：

```
openssl req -x509 -new -nodes -sha256 -utf8 -days 3650 -newkey rsa:2048 -keyout server.key -out server.crt -config ssl.conf
```

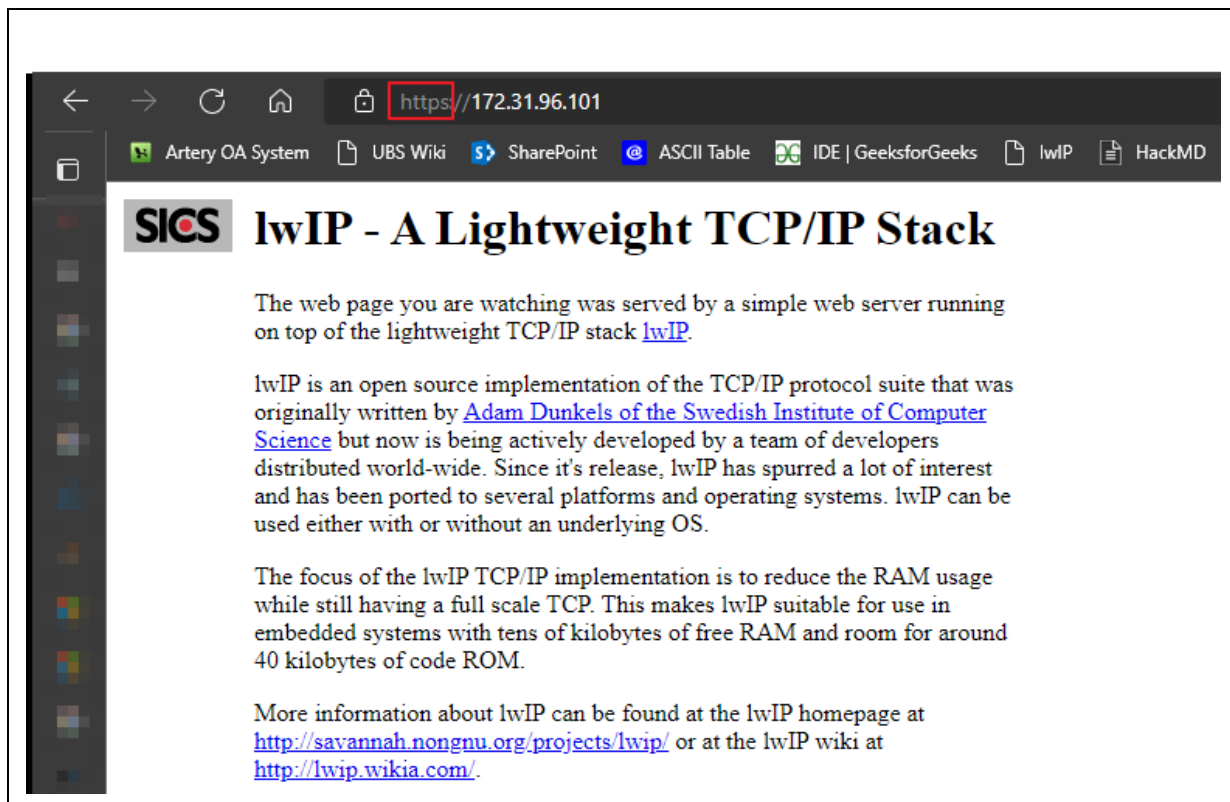
生成的私钥与凭证档案会跟 ssl.conf 在同一个目录下。

3. 汇入自签凭证到「受信任的跟证书授权单位」
光是建立好自签凭证还是不够的，网站服务器也设定正确才行，这毕竟是一个 PKI 基础架构，你还必须让所有需要安全联机的端点都能互相信任才行，因此你还须将建立好的自签凭证安装到「受信任的跟证书授权单位」之中，这样子你的操作系统或浏览器才能将你的自签凭证视为「可信任的联机」。以下为手动汇入的步骤：
 - 开启档案总管，并鼠标双击 server.crt 档案
 - 点击「安装凭证」按钮
 - 选取「目前使用者」并按「下一步」继续
 - 选取「将所有凭证放入以下的存放区」并按下「浏览」按钮
 - 选取「受信任的跟证书授权单位」并按下「确定」
 - 按「下一步」继续
 - 按「完成」继续
 - 在 安全性警告 窗口按下「是(Y)」即可完成设定
4. 将私钥及凭证汇入到 TLS server
TLS server 负责解密数据，在本应用笔记中就是 MCU 端，分别将私钥及凭证填入 demo code 中的 mbedtls_test_srv_key 及 mbedtls_test_srv_cert 即可。

2.4 实验效果

- 浏览器的网址以 HTTPS 开头，且锁头图示为上锁的状态

图 3. HTTPS 页面



3 文档版本历史

表 1. 文档版本历史

日期	版本	变更
2022.9.19	2.0.0	最初版本
2023.8.8	2.0.1	加入FreeRTOS来管理各个Task之间的排程

重要通知 - 请仔细阅读

买方自行负责对本文所述雅特力产品和服务的选择和使用，雅特力概不承担与选择或使用本文所述雅特力产品和服务相关的任何责任。

无论之前是否有过任何形式的表示，本文档不以任何方式对任何知识产权进行任何明示或默示的授权或许可。如果本文档任何部分涉及任何第三方产品或服务，不应被视为雅特力授权使用此类第三方产品或服务，或许可其中的任何知识产权，或者被视为涉及以任何方式使用任何此类第三方产品或服务或其中任何知识产权的保证。

除非在雅特力的销售条款中另有说明，否则，雅特力对雅特力产品的使用和/或销售不做任何明示或默示的保证，包括但不限于有关适销性、适合特定用途(及其依据任何司法管辖区的法律的对应情况)，或侵犯任何专利、版权或其他知识产权的默示保证。

雅特力产品并非设计或专门用于下列用途的产品：(A) 对安全性有特别要求的应用，如：生命支持、主动植入设备或对产品功能安全有要求的系统；(B) 航空应用；(C) 汽车应用或汽车环境；(D) 航天应用或航天环境，且/或(E) 武器。因雅特力产品不是为前述应用设计的，而采购商擅自将其用于前述应用，即使采购商向雅特力发出了书面通知，风险由购买者单独承担，并且独力负责在此类相关使用中满足所有法律和法规要求。

经销的雅特力产品如有不同于本文档中提出的声明和/或技术特点的规定，将立即导致雅特力针对本文所述雅特力产品或服务授予的任何保证失效，并且不应以任何形式造成或扩大雅特力的任何责任。

© 2022 雅特力科技 保留所有权利